TEMA 4 · GRAMÁTICAS INDEPENDIENTES DEL CONTEXTO

Formas normales

Chomsky y Greibach, paso a paso

Basado en el Tema 4 de Serafín Moral, Universidad de Granada.
Dos ejemplos nuevos y su relación con la pertenencia.

Nuria Rodríguez Barroso

Qué significa normalizar una gramática

Imponemos un patrón a todas las producciones para facilitar el trabajo con la gramática.

$$L(G_{\mathrm{Ch}}) = L(G_{\mathrm{Gr}}) = L(G)$$

Cambian las reglas y pueden aparecer variables auxiliares. Las palabras generadas se conservan.

> Trabajaremos con lenguajes que no contienen ε. La palabra vacía se trata mediante una excepción para el símbolo inicial.

Dos patrones para las producciones

Chomsky

$$A \to BC \quad \text{o} \quad A \to a$$

Dos variables o un terminal.

Greibach

$$A \to a\alpha, \quad \alpha \in V^*$$

Un terminal, seguido de cero o más variables.

> B y C son variables. a es un terminal. En Greibach también se permite A → a.

Preparación de la gramática

Símbolos útiles. Eliminamos lo no productivo y después lo inalcanzable.

Producciones nulas. Las eliminamos y tratamos ε por separado.

Producciones unitarias. Las sustituimos por reglas no unitarias.

> Los dos ejemplos ya están preparados: no tienen símbolos inútiles ni producciones nulas o unitarias.

Reglas válidas en Chomsky

Producción	¿Válida?	Motivo
S → AB	Sí	Dos variables
A → a	Sí	Un terminal
S → aB	No	Mezcla terminal y variable
S → ABC	No	Tres variables

Ejemplo 1: la gramática inicial

$$S \to aSb \mid ab \mid c$$

Inicial: S

Terminales: {a, b, c}

Una única variable: S.

> Cada uso de S → aSb añade una a al principio y una b al final.

Qué palabras genera el ejemplo 1

$$L_1 = \{a^n b^n : n \geq 1\} \cup \{a^n c b^n : n \geq 0\}$$

Terminamos con ab

$S \Rightarrow ab$

$S \Rightarrow aSb \Rightarrow aabb$

Terminamos con c

$S \Rightarrow c$

$S \Rightarrow aSb \Rightarrow acb$

> ε no pertenece a este lenguaje. La transformación debe conservar ambas familias de palabras.

Qué tenemos que transformar

$S \to aSb$

$S \to ab$

$S \to c$

aSb: mezcla símbolos y tiene longitud 3.

ab: son dos terminales.

S → c ya es válida.

Primero sustituimos los terminales de las reglas largas. Después reducimos su longitud.

Variables auxiliares para a y b

$$C_a \to a$$
$$C_b \to b$$

C_a representa una a.

C_b representa una b.

Las dos reglas están en Chomsky.

C_a y C_b son variables nuevas. Podemos reutilizarlas en todas las producciones.

Sustituimos los terminales de aSb

$S \to aSb$

$C_a \to a$

$C_b \to b$

a se sustituye por C_a.

b se sustituye por C_b.

$S \to C_aSC_b$

La S central permanece. C_a y C_b recuperarán los terminales originales.

Sustituimos los terminales de ab

$S \to ab$

$S \to c$

$C_a \to a$

$C_b \to b$

S → ab pasa a **$S \to C_a C_b$**.

S → c permanece igual.

No sustituimos c por una variable: S → c ya cumple el patrón de Chomsky.

Solo queda una regla larga

$S \to C_a S C_b \mid C_a C_b \mid c$

$C_a \to a$

$C_b \to b$

$S \to C_a S C_b$ tiene tres variables.

Necesitamos dividirla en reglas con dos variables.

$S \to C_a C_b$, $S \to c$, $C_a \to a$ y $C_b \to b$ ya son válidas.

Dividimos C_aSC_b en dos pasos

$$S \to C_aD_1$$
$$D_1 \to SC_b$$

D_1 es una variable nueva.

$S \Rightarrow C_aD_1 \Rightarrow C_aSC_b$

D_1 agrupa el sufijo SC_b. Ambas producciones tienen exactamente dos variables.

Resultado en forma normal de Chomsky

$$S \to C_a D_1 \mid C_a C_b \mid c$$

$$D_1 \to S C_b$$

$$C_a \to a$$

$$C_b \to b$$

Dos variables:

$S \to C_a D_1$, $S \to C_a C_b$, $D_1 \to S C_b$

Un terminal:

$S \to c$, $C_a \to a$, $C_b \to b$

No hay producciones nulas ni unitarias. Todas las reglas cumplen uno de los dos patrones.

La misma palabra con las nuevas reglas

La palabra **acb** se obtiene en las dos gramáticas.

Antes

$$S \Rightarrow aSb \Rightarrow acb$$

En Chomsky

$$S \Rightarrow C_aD_1 \Rightarrow aD_1 \Rightarrow aSC_b \Rightarrow acC_b \Rightarrow acb$$

> La palabra no cambia. La derivación utiliza más pasos porque las reglas son más pequeñas.

Por qué se conserva todo el lenguaje

Cada aplicación de $S \to aSb$ se simula con $S \to C_a D_1$, $D_1 \to SC_b$, $C_a \to a$ y $C_b \to b$.

$S \to ab$ se simula con $S \to C_a C_b$ y las reglas de C_a y C_b. $S \to c$ no cambia.

En sentido inverso, D_1 solo puede convertirse en SC_b, C_a en a y C_b en b. Las auxiliares no añaden opciones nuevas.

> Comparar una sola palabra ilustra la transformación. La correspondencia entre reglas justifica la equivalencia completa.

El procedimiento de Chomsky

Terminales en reglas largas. Sustituimos cada terminal a por una variable nueva C_a, con la regla $C_a \to a$.

Más de dos variables. Dividimos cada lado derecho largo usando auxiliares nuevas para esa producción.

$$A \to BEFG \quad \rightsquigarrow \quad A \to BD_1,\ D_1 \to ED_2,\ D_2 \to FG$$

> Las reglas que ya son de la forma A → a se conservan.

Reglas válidas en Greibach

Producción	¿Válida?	Motivo
S → aAB	Sí	Terminal seguido de variables
A → a	Sí	Cero variables después
S → AB	No	Empieza por una variable
S → abB	No	El segundo símbolo es terminal

Ejemplo 2: partimos de Chomsky

$A1 \to A1A3 \mid A2A3$

$A3 \to a$

$A2 \to b$

Inicial: A_1

Orden: A_1, A_2, A_3

Todas las reglas ya están en Chomsky.

Las reglas de A_1 empiezan por una variable.

A_3ún no están en Greibach.

La recursión $A_1 \to A_1A_3$ es por la izquierda: el lado derecho empieza por la propia A_1.

Qué palabras genera el ejemplo 2

$$A_1 \Rightarrow A_1A_3 \Rightarrow A_1A_3A_3 \Rightarrow A_2A_3A_3A_3 \Rightarrow bA_3A_3A_3 \Rightarrow^* baaa$$

$$L_2 = \{ba^n : n \geq 1\}$$

Cada uso de $A_1 \rightarrow A_1A_3$ deja una A_3 adicional a la derecha.

Para terminar usamos $A_1 \rightarrow A_2A_3$. Por eso siempre aparece una b y al menos una a.

Las dos operaciones de Greibach

ELIMINA1 (A → Bα). Si una regla empieza por B, reemplazamos esa B por cada alternativa de B.

ELIMINA2 (A). Reorganizamos las reglas A → Aα mediante la variable auxiliar B_A.

> En este ejemplo eliminaremos primero la recursión de A_1 y después sustituiremos las variables iniciales.

La repetición que debemos conservar

$A1 \rightarrow A1A3 \mid A2A3$

Sin repetición: A_2A_3

Una repetición: $A_2A_3A_3$

Dos repeticiones: $A_2A_3A_3A_3$

A_1 genera A_2A_3 seguido de cero o más A_3. La nueva gramática debe conservar exactamente esas posibilidades.

Una auxiliar para las A3 adicionales

$$B_1 \to A_3 \mid A_3 B_1$$

B_1 genera una o más A_3.

$B_1 \Rightarrow A_3$

$B_1 \Rightarrow A_3 B_1 \Rightarrow A_3 A_3$

B_1 es la auxiliar creada por ELIMINA2(A_1). No genera ε. La alternativa de cero repeticiones se mantendrá directamente en A_1.

A1 deja de empezar por sí misma

$A_1 \to A_2A_3 \mid A_2A_3B_1$

$B_1 \to A_3 \mid A_3B_1$

$A_3 \to a$

$A_2 \to b$

$A_1 \to A_2A_3$: ninguna A_3 adicional.

$A_1 \to A_2A_3B_1$: una o más A_3 adicionales.

ELIMINA2(A_1): retiramos $A_1 \to A_1A_3$. El lenguaje sigue siendo $\{ba^n : n \geq 1\}$.

El esquema de la transformación

Antes

$$A \to A\alpha \mid \beta$$

La alternativa β permite salir de la recursión.

Después

$$A \to \beta \mid \beta B_A$$

$$B_A \to \alpha \mid \alpha B_A$$

> Aquí α no es vacía y β no empieza por A. En el ejemplo, la variable recursiva es A_1, α = A_3 y β = A_2A_3. La auxiliar es B_1.

Sustituimos A2 al principio de A1

$A1 \longrightarrow A2A3 \mid A2A3B1$

$A2 \longrightarrow b$

$A_1 \rightarrow A_2A_3$ pasa a $\mathbf{A_1 \rightarrow bA_3}$.

$A_1 \rightarrow A_2A_3B_1$ pasa a $\mathbf{A_1 \rightarrow bA_3B_1}$.

$A_1 \Rightarrow A_2A_3 \Rightarrow bA_3$ y $A_1 \Rightarrow A_2A_3B_1 \Rightarrow bA_3B_1$. Aplicamos ELIMINA1 a las dos reglas de A_1.

Sustituimos A3 al principio de B1

$B1 \longrightarrow A3 \mid A3B1$

$A3 \longrightarrow a$

$B_1 \rightarrow A_3$ pasa a $\mathbf{B_1 \rightarrow a}$.

$B_1 \rightarrow A_3B_1$ pasa a $\mathbf{B_1 \rightarrow aB_1}$.

Aplicamos ELIMINA1. La regla unitaria $B_1 \rightarrow A_3$ también desaparece.

Resultado en forma normal de Greibach

$$A_1 \to bA_3 \mid bA_3B_1$$

$$B_1 \to a \mid aB_1$$

$$A_3 \to a$$

Cada regla empieza por un terminal.

Después solo aparecen variables.

A_2 ha quedado inalcanzable y se elimina.

$A_1 \to bA_3$ genera ba. $A_1 \to bA_3B_1$ genera b seguida de al menos dos a.

La misma palabra: baaa

Gramática original en Chomsky

$$A_1 \Rightarrow A_1A_3 \Rightarrow A_1A_3A_3 \Rightarrow A_2A_3A_3A_3$$
$$\Rightarrow bA_3A_3A_3 \Rightarrow baA_3A_3 \Rightarrow baaA_3 \Rightarrow baaa$$

Gramática en Greibach

$$A_1 \Rightarrow bA_3B_1 \Rightarrow baB_1 \Rightarrow baaB_1 \Rightarrow baaa$$

> Con n = 4: Chomsky necesita 7 pasos y Greibach 4. Ambas generan la misma palabra.

Sustitución con varias alternativas

$$A \to B\alpha, \qquad B \to \beta_1 \mid \beta_2$$

$$A \to \beta_1\alpha \mid \beta_2\alpha$$

ELIMINA1 (A → Bα): cada alternativa de B da lugar a una nueva producción de A.

> Hay que copiar todas las alternativas. El sufijo α se conserva en cada una.

En gramáticas más grandes

Las variables pueden depender unas de otras y formar recursiones indirectas.

Ordenamos $A_1, \ldots, A_m$. Aplicamos ELIMINA1 a $A_k \to A_j \alpha$ cuando $j < k$.

Si aparece $A_k \to A_k \alpha$, aplicamos ELIMINA2(A_k) y creamos B_k.

Volvemos en orden inverso con ELIMINA1 para las reglas $A_i \to A_j \alpha$ ($j > i$). Después tratamos $B_j \to A_i \alpha$.

> Chomsky ofrece un punto de partida adecuado, aunque no es obligatorio pasar por ella para llegar a Greibach.

Pertenencia: profundidad de búsqueda

Para una palabra de longitud n ≥ 1, contamos aplicaciones de producciones.

Chomsky

$$2n - 1$$

n reglas a terminales y n − 1 reglas binarias.

Greibach

$$n$$

Cada aplicación introduce exactamente un terminal.

> Es la profundidad de la búsqueda de derivaciones. No es la altura del árbol de derivación.

Por qué Chomsky sigue siendo útil

$$A \to BC, \qquad w = uv$$

Para saber si A genera w, buscamos una partición donde B genere u y C genere v.

El algoritmo CYK guarda qué variables generan cada fragmento y reutiliza esos resultados.

La estructura binaria permite programación dinámica, sin enumerar todas las derivaciones.

> Greibach reduce la profundidad de una búsqueda exhaustiva. Eso no garantiza menor tiempo de ejecución.

Un paso de CYK con el ejemplo 1

Para reconocer **acb** con la gramática en Chomsky:

Fragmento	Variable	Justificación
a	C_a	$C_a \to a$
c	S	$S \to c$
b	C_b	$C_b \to b$
cb	D_1	$D_1 \to SC_b$ combina c y b
acb	S	$S \to C_aD_1$ combina a y cb

> S genera el fragmento completo: acb pertenece al lenguaje.

La palabra vacía como excepción

Las formas descritas no pueden generar ε: ninguna producción elimina todos los símbolos.

Si ε pertenece al lenguaje original, se permite una excepción $S_0 \to \varepsilon$ con un inicial nuevo que no aparezca en lados derechos.

> Las demás producciones mantienen el patrón correspondiente. Las cotas n y 2n − 1 se aplican a palabras no vacías.

Qué cambia y qué se conserva

Aspecto	Chomsky	Greibach
Regla	$A \to BC$ o $A \to a$	$A \to a\alpha$, con $\alpha \in V^*$
Estructura	Descomposición binaria	Un terminal por paso
Uso destacado	Algoritmo CYK	Derivaciones y autómatas con pila
Lenguaje	Se conserva, con ε tratada por separado	

> Una forma normal no garantiza que la gramática sea no ambigua.

RESUMEN

Las ideas que nos llevamos

Chomsky. Sustituimos terminales de reglas largas y dividimos lados derechos en pares.

Greibach. Eliminamos recursión por la izquierda y sustituimos variables iniciales hasta empezar por un terminal.

Pertenencia. Distinguimos la longitud de una derivación de la eficiencia del algoritmo que la busca.

Referencia conceptual: Serafín Moral, Tema 4, Universidad de Granada. Ejemplos nuevos y desarrollo docente: Nuria Rodríguez Barroso.